

HW2答案

(30) 1. 写出十进制数63.25的二进制表达。要求分别采用IEEE 754 单精度和双精度格式。

- [illegible]

转换为二进制

- 整数部分 $63 = 111111_2$ (因为 $63 = 32+16+8+4+2+1$, 即 $0b111111$)
- 小数部分 $0.25 = 0.01_2$ (因为 $0.25 = 2^{-2}$)
- 故 $63.25 = 111111.01_2$

规格化

- $111111.01 = 1.111101 \times 2^5$ (移动小数点5位, 指数=5)

IEEE 754 单精度 (32位)

- 符号位 $S = 0$ (正数)
- 指数位 $E = 5 + 127 = 132 = 10000100_2$ (8位)
- 有效数字位 M (去掉整数部分的1): 1111101 后补0至23位 →
11111010000000000000000
- 完整32位: 0 10000100 11111010000000000000000
- 十六进制: 0x427D0000 (0100 0010 0111 1101 0000 0000 0000 0000)

IEEE 754 双精度 (64位)

- [illegible]

知识点

■ IEEE 754规定:

单精度浮点数的偏阶为127, 即: ***Bias=127***

双精度浮点数的偏阶为1023, 即: ***Bias=1023***

[IEEE 754浮点数标准详解 - C语言中文网](#)

(30) 2. 给出下面指令的类型和十六进制表示: `sw $t1, 32($t2)`

ppt71页开始

- **指令含义:** 将寄存器 `$t2` 的值与立即数 `32` 相加, 得到目标内存地址。将寄存器 `$t1` 中的 32 位数据存入该内存地址。 `Memory[$t2 + 32] = $t1`
 - **指令类型:** I, 因为使用基址偏移寻址。
 - **十六进制表示:**
 - 操作码 `op = 43` (sw 的 opcode = $43_{10} = 0x2B = 101011_2$)
 - 寄存器编号:
 - `rs = $t2 = 1010 (0x0A) = 010102`
 - `rt = $t1 = 910 (0x09) = 010012`
 - 立即数 `32 = 3210 = 0x0020 = 00000000001000002`
 - 机器码 (32位): `101011 01010 01001 0000000000100000`
 - 分组: 1010 1101 0100 1001 0000 0000 0010 0000
 - 十六进制: `0xAD490020`
- op:6bit, rs 5bit, rt 5bit, 立即数: 16bit

I类型

I-Type	Op	Rs	Rt	16 bit Address or Immediate

说明:

- 用于短立即数和内存载入指令load操作
- 立即数, 顾名思义, 就是可以立即使用的数, 即在指令中就给了具体的数据, 而不用先给出寄存器号到寄存器中去找

参数说明:

- Op: 指令操作码
- Rs: 第一个源操作数寄存器号, 参与运算使用
- Rt: 第二个源操作数寄存器号, 参与运算使用
- 16位立即数: 作为数据, 参与运算使用

(40) 3. 将下面的函数f翻译成MIPS汇编语言。假设函数func的声明为“int func (int a, int b);”，函数f的代码如下：

```
int f ( int a, int b, int c, int d) {  
    return func (func (a, b), c+d);  
}
```

答案

```
f: addi    $sp, $sp, -12  
   sw     $ra, 8($sp)  
   sw     $s1, 4($sp)  
   sw     $s0, 0($sp)  
   move   $s1, $a2  
   move   $s0, $a3  
   jal    func  
   move   $a0, $v0  
   add    $a1, $s0, $s1  
   jal    func  
   lw     $ra, 8($sp)  
   lw     $s1, 4($sp)  
   lw     $s0, 0($sp)  
   addi   $sp, $sp, 12  
   jr     $ra
```

a, b, c, d — — \$a0, \$a1, \$a2, \$a3

1

```
f:  
    addi $sp, $sp, -12 # 分配栈空间  
  
    sw   $ra, 8($sp) # 保存返回地址  
    sw   $a2, 4($sp)
```

```

sw    $a3, 0($sp)
# func(a,b)
jal   func

# 第二次调用参数
move  $a0, $v0

lw     $t0, 4($sp)
lw     $t1, 0($sp)
add    $a1, $t0, $t1

# func(func(a,b), c+d)
jal   func

lw     $ra, 8($sp)
addi   $sp, $sp, 12
jr     $ra

```

2

```

f:
    addi    $sp, $sp, -8
    sw      $ra, 4($sp)
    sw      $s0, 0($sp)

    add     $s0, $a2, $a3      # $s0 = c + d

    jal     func              # 第一次调用
    move    $a0, $v0          # 第一次返回值作为第二次调用的第一个参数
    move    $a1, $s0

    jal     func              # 第二次调用

    lw      $s0, 0($sp)
    lw      $ra, 4($sp)
    addi    $sp, $sp, 8
    jr      $ra

```

```
f:
    addi    $sp, $sp, -8
    sw      $ra, 4($sp)

    add     $t0, $a2, $a3      # $s0 = c + d
    sw     $t0, 0($sp)

    jal     func               # 第一次调用
    move    $a0, $v0           # 第一次返回值作为第二次调用的第一个参数
    lw      $a1, 0($sp)

    jal     func               # 第二次调用

    lw      $ra, 4($sp)
    addi    $sp, $sp, 8
    jr      $ra
```

易错点

- 需要保存 `$ra`，调用函数会覆盖返回地址：
 - 第一次 `jal func`： `$ra` 被设置为第一次调用后需要返回的地址（即 `move $a0, $v0` 指令的地址）。这个地址用于在 `func` 执行完毕后返回到 `f` 函数中继续执行。
 - 第二次 `jal func`：再次执行 `jal` 时， `$ra` 又被写入新地址（即 `lw $ra, 4($sp)` 指令的地址）。
- 但第一次 `jal` 后， `$a2`， `$a3` 可能已失效，严格来说应提前保存。
- 第一次调用后，返回值在 `$v0`。
- 第二次调用参数顺序错误。

错误案例：

```
f:
    addi $sp, $sp, -4
    sw   $ra, 0($sp)

    jal func                # 第一次调用 func(a,b)，假设a,b已在$a0,$a1
    add  $a1, $a2, $a3      # $a1 = c+d
    move $a0, $v0           # 第一次返回值作为新$a0
    jal func                # 第二次调用
```

```
lw    $ra, 0($sp)
addi $sp, $sp, 4
jr    $ra
```

知识点参考

MIPS的操作数

名称	寄存器号	用途	调用时是否保存
\$zero	0	常数0	n.a.
\$v0~\$v1	2~3	计算结果和表达式求值	否
\$a0~\$a3	4~7	参数	否
\$t0~\$t7	8~15	临时变量	否
\$s0~\$s7	16~23	保存的寄存器	是
\$t8~\$t9	24~25	更多临时变量	否
\$gp	28	全局指针	是
\$sp	29	栈指针	是
\$fp	30	帧指针	是
\$ra	31	返回地址	是

图 2-14 MIPS 寄存器约定。称为 \$at 的 1 号寄存器被汇编器所保留（见 2.12 节），称为 \$k0~\$k1 的 26~27 号寄存器被操作系统所保留。关于这点也可见 MIPS 参考数据卡的第 2 列

■ 注：MIPS对18个寄存器是否被保存作如下分组：

\$t0 ~ \$t9: 10 个临时寄存器，在过程调用中**不必被调用者保存**。

\$s0 ~ \$s7: 8 个保留寄存器，**必须被保存**。

过程调用时，保留和不保留的内容：

保留	不保留
保存寄存器：\$s0 ~ \$s7	临时寄存器：\$t0 ~ \$t9
栈指针寄存器：\$sp	参数寄存器：\$a0 ~ \$a3
返回地址寄存器：\$ra	返回值寄存器：\$v0 ~ \$v1
栈指针以上的栈	栈指针以下的栈

2.2.2 MIPS的操作数

小结:

MIPS operands

Name	Example	Comments
32 register	\$s0,\$s1..... \$t0, \$t1.....	Fast locations for data. In MIPS, data must be in registers to perform arithmetic.
2 ³⁰ memory words	Memory[0], Memory[4], Memory[8],, Memory[4294967292]	Accessed only by data transfer instructions in MIPS. MIPS uses byte addr., so sequential word addr. Differ by 4. Memory holds data structures, arrays, and spilled registers.

MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add \$s1,\$s2,\$s3	$\$s1 = \$s2 + \$s3$	Three register operands
	subtract	sub \$s1,\$s2,\$s3	$\$s1 = \$s2 - \$s3$	Three register operands
	Add immediate	addi \$s1,\$s2,100	$\$s1 = \$s2 + 100$	Used to add constants
Data transfer	load word	lw \$s1, 100(\$s2)	$\$s1 = \text{Memory}[\$s2 + 100]$	Data from memory to register
	store word	sw \$s1, 100(\$s2)	$\text{Memory}[\$s2 + 100] = \$s1$	Data from register to memory
Uncondition-al jump	jump	j L	go to L	Jump to target address
	jump register	jr \$ra	go to \$ra	For procedure return

□ 无条件跳转

- **jump** j 2500
- **jump register** jr \$ra
- **jump and link** jal 2500

如: `move $t0,$t1`

register \$t0 gets register \$t1

11

```
add $t0,$zero, $t1 # register $t0 gets 0+register $t1
```

```
# 调用实例

.data

    local_var: .word 0

.text

main:

    # 调用函数
    jal func_name

    # ...

    # 返回
    jr $ra
```

```
func_name:
    # 保存调用者环境
    addi $sp, $sp, -4
    sw $ra, 0($sp)
    # ...
    # 赋值局部变量
    li $t0, 10
    sw $t0, local_var
    # ...
    # 恢复调用者环境
    lw $ra, 0($sp)
    addi $sp, $sp, 4
    jr $ra
```

评判标准

前两题标准HW1。做错误题目的扣完，有过程给5分鼓励。第三题错一处（或一个知识点）扣5分。再次基础上，抄作业明显抄错或者严重的基础知识缺失扣10分（例如，主=6），不会超出题目失分上限。